

Right decrement triangle

→

↓ rows = i

col = j

0	*	*	*	*	*
1		*	*	*	*
2			*	*	*
3				*	*
4					*

i	value	j - runs for
0	→ 5	= $n-i$
1	→ 4	= $n-i = 5-1 = 4$
2	→ 3	
3	→ 2	
4	→ 1	

i	j	→	i	value	spaces	Stars
0	→	* * * * *	→	0	0	5
1	→	- * * * *	→	1	1	4
2	→	- - * * *	→	2	2	3
3	→	- - - * *	→	3	3	2
4	→	- - - - *	→	4	4	1

(n-i)

```

1 n = 5
2 for i in range(n):
3     for k in range(i):
4         print(" ", end=" ")
5     → for j in range(n-i):
6         print("#", end=" ")
7     print()

```

```

# # # # #
- # # # #
- - # # #

```

Right decrement triangle

Diagram showing row and column indices:

rows = i (vertical arrow pointing down)

col = j (horizontal arrow pointing right)

	0	1	2	3	4
0					*
1				*	*
2			*	*	*
3		*	*	*	*
4	*	*	*	*	*

i	Spa	*
0	4	1
1	3	2
2	2	3
3	1	4
4	0	5

$(n-i-1)$ $(i+1)$

n=5

	0	1	2	3	4	5	6	7	8
0 →					*				
1 →				*	*	*			
2 →			*	*	*	*	*		
3 →		*	*	*	*	*	*	*	
4 →	*	*	*	*	*	*	*	*	*

i	Space	*
0	4	1 } 2
1	3	3 } 2
2	2	5 } 2
3	1	7 } 2
4	0	9

$(n-i-1)$

x=1

```

for i in range(5):
    for k in range(n-i-1):
        space
    for i in range(x):
        *
    
```

x+=2

2nd approach

i	*
0 →	1 } 2
1 →	3 } 2
2 →	5 } 2
3	7 } 2
4	9

$(i * 2 + 1) \rightarrow 0 * 2 + 1 \rightarrow 0 + 1 \Rightarrow \textcircled{1}$
 $1 * 2 + 1 \rightarrow 2 + 1 \rightarrow \textcircled{3}$
 $2 * 2 + 1 \rightarrow 4 + 1 \rightarrow \textcircled{5}$
 $\textcircled{7}$
 $\textcircled{9}$

$$x = 135$$

$$5 - 2 - 1 \Rightarrow 2$$

→ for i in range(5):

→ for k in range(n-i-1):
space

→ for i in range(x)
*

$$x += 2$$

```

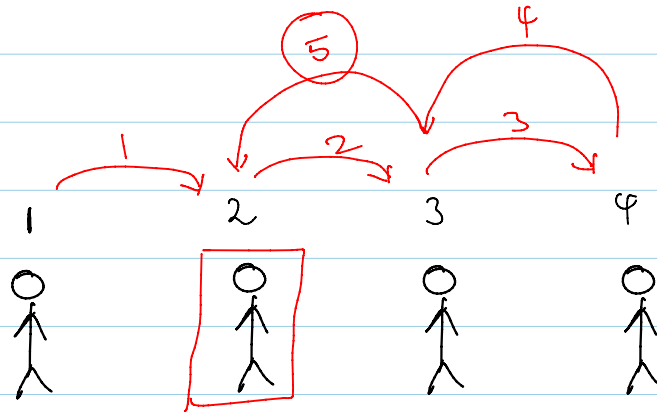
_ _ _ _ *
_ _ _ * * *
_ _ * * * *

```

Pass the pillow:

$$n = 4$$

$$p = 5$$



$$n = ?$$

$$p = ?$$

$$n = 4$$

$$p = 5$$

$$\begin{aligned} & \begin{cases} d \\ p \end{cases} \quad \begin{aligned} & = -1 \\ & = 1 \end{aligned} \\ & \quad \quad \quad = 1, 1, 1, \boxed{-1}, 1 \end{aligned}$$

$$dir = +1$$

$$pers = 1$$

$$4 + (-1)$$

$$4 - 1 = \boxed{3}$$

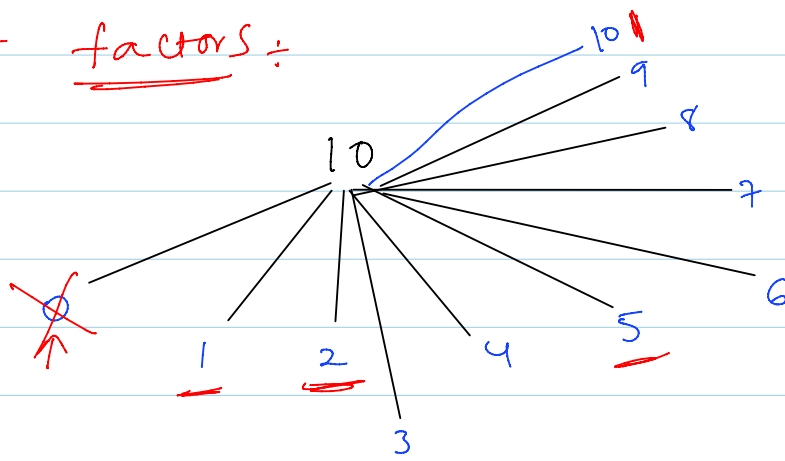
for i in range(p)

person += dir

if (p == n)

dir = -1

Print factors:

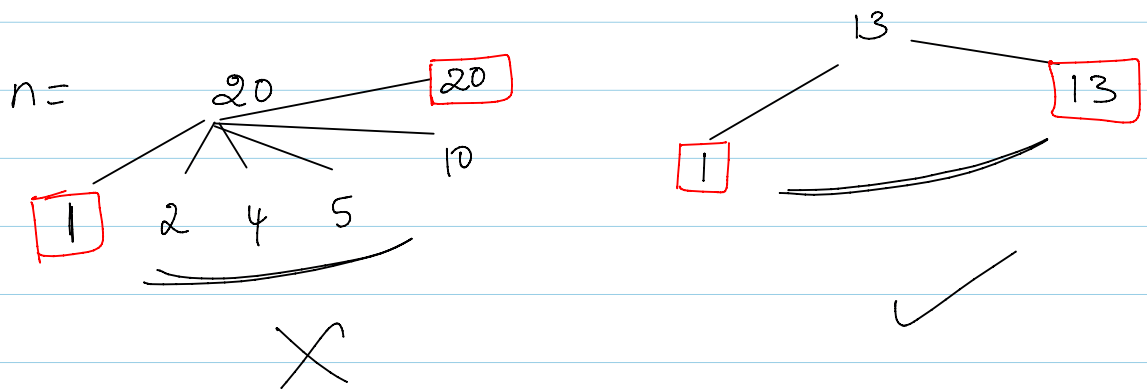


```
for i in range(1, n+1)
```

```
if (n % i == 0)
```

```
    print(i)
```

Prime Number check:



✗ 1 2 4 5 10 20 ✗

→ if we have any factors other than number and 1 it is not prime number else it is a prime number

```
1  n = 13
2  flag = False;
3  for i in range(2,n):
4      if(n % i == 0):
5          flag = True
6          break
7  if flag == True:
8      print("Not Prime")
9  else:
10     print("Prime")
```

problems up on numbers:-

num = 1024

operations:- $(1 + 0 + 2 + 4) \Rightarrow 7$

←
1024

% 2 $\rightarrow (0, 1)$

% 3 $\rightarrow (0, 1, 2)$

% 4 $\rightarrow (0, 1, 2, 3)$

% 5 $\rightarrow (0, 1, 2, 3, 4)$

⋮

% 10 $\rightarrow (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)$

✓ 1024 % 10 $\rightarrow$ 4

$\Rightarrow$ 1024 // 10 $\Rightarrow$ 102

✓ 102 % 10 $\rightarrow$ 2

$\rightarrow$ 102 // 10 $\rightarrow$ 10

✓ 10 % 10 $\rightarrow$ 0

$\rightarrow$ 10 // 10 $\rightarrow$ 1

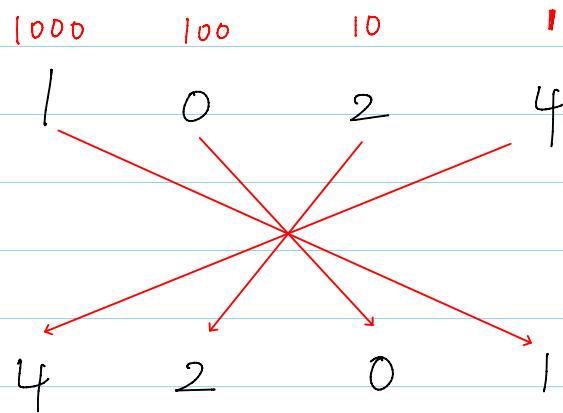
1 % 10 $\rightarrow$ 1

$\rightarrow$ 1 // 10 $\rightarrow$ 0

```
1 num = 1024
2 sum = 0
3 while(num > 0):
4     last = num % 10
5     sum += last
6     num = num // 10
7 print(sum)
```

Reverse Number

$$\text{num} = 1024 \Rightarrow 4201$$



$$4 \times 1000 + 2 \times 100 + 0 \times 10 + 1 \times 1$$

$$4000 + 200 + 0 + 1 \Rightarrow [4201]$$

$$\text{num} = 1024$$

$$\text{rev} = 0$$

while (num > 0):

$$\text{last} = \text{num} \% 10$$

$$\text{rev} = \text{rev} * 10 + \text{last}$$

$$\text{num} = \text{num} // 10$$

$$4 \times 1000 + 2 \times 100 + 0 \times 10 + 1 \times 1$$

$$\begin{array}{cccc} \downarrow & / & \downarrow & \downarrow \\ 1 \times 10 \times 10 \times 10 & 1 \times 10 \times 10 & 1 \times 10 & \underline{1 \times 1} \end{array}$$

```

1 num = 1024 102 10 1
2 rev = 0442 420 420 4201
3 while(num > 0):
4     last = num % 10 → 1
5     rev = rev * 10 + last → 420 × 10 + 1
6     num = num // 10
7 print(rev)

```

rev = 4

→ $0 \times 10 + 4$ → $0 \times 10 + 4$

→ $4 \times 10 + 2$ → $4 \times 10 + 2$

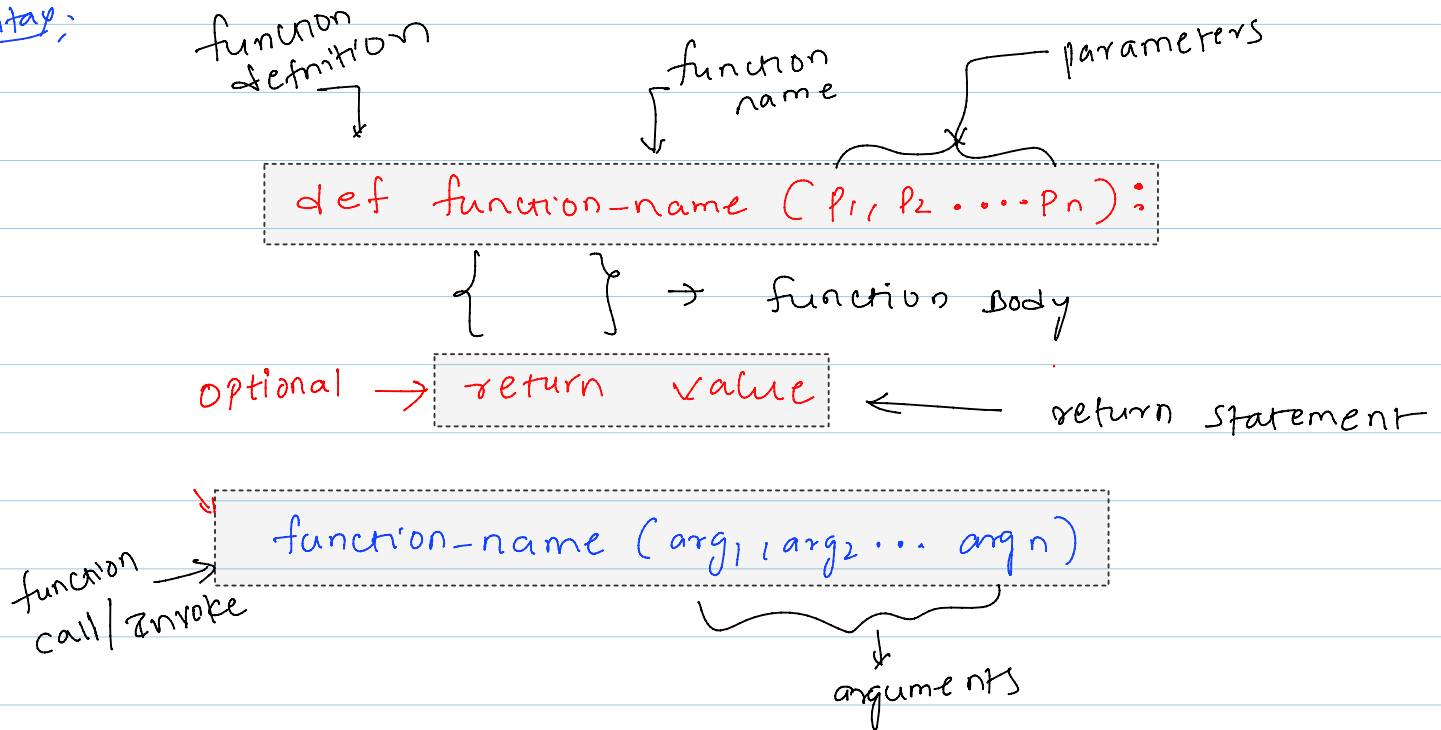
→ $42 \times 10 + 0$ → $42 \times 10 + 0$

$420 \times 10 + 1$ → $420 \times 10 + 1$

Introduction to functions

→ a function is a block of reusable code, that we can call any number of times

Syntax:



```
def left_triangle ( n ) :
```

```
    for i in range(n):  
        for j in range(i+1):  
            print("*", end=" ")  
        print()
```

```
left_triangle(5)
```

Types of Parameters and Arguments:

$\text{def } f(P) \leftarrow \text{Parameter}$

$f(10) \leftarrow \text{Argument}$

1) Normal parameters :-

```
def sum(a, b):  
    print(a + b)
```

2) Default parameters

```
def chrome(name = "Guest"):  
    print("welcome", name)
```

3) *args argument :-

→ It will accept any number of Arguments

```
def addAll(*args):  
    print(args)
```

```
print(1, 2, 3, sep = " ", end = "\n")
```

```
def print(*args, sep = "", end = "\n") :
```

//logrc

1) Normal Arguments;

`fn(10, 20, 30)`

2) positional Arguments

`fn(name = "Pavan", age = 20)`

Return in functions

`def fn():`

`None` ← `return val` ← optional

→ by default a function will return `None`

→ if we return something `None` will be overridden

```
1 def sum(a,b):  
2     return a+b  
3  
4 x = sum(10,20)  
5  
6 print(x+20)
```


Introduction to Strings :-

→ Strings are sequence of characters enclosed b/w single, double, triple quotes

`s = str("Hello")`

`s = hello`

"Hello"

'Hello'

"""Hello"""

Mutable vs Immutable

→ strings are immutable (once you created a string can't modify content but re assign)

String Methods :-

`s = "Hello"`

`upper()` → function

1) `s.upper()` → Convert to upper case

2) `s.lower()` → Convert to lower case

3) `s.capitalize()` → first letter capital

4) `s.find(str)` → finds index of str in s

5) `s.count(str)` → counts occurrence of str in s

6) `s.split()` → split to list

7) `join()` → convert to string

String Indexing and Slicing

→ we have 2 types of indexing +ve and -ve

s = "Hello"

	0	1	2	3	4	
MSB	H	e	l	l	O	LSB
	-5	-4	-3	-2	-1	

`s[0]` → H

`s[-2]` → l

} we can use +ve and -ve indexing on strings to get characters

*** Slicing operator

$s[start: end: step] \leftrightarrow range(start, end, step)$

0, 4, 1
0 1 2 3

$s = \text{hello}$

	0	1	2	3	4	
MSB	h	e	l	l	o	LSB
	-5	-4	-3	-2	-1	

$s[0:4:1]$ $range(0, 4, 1) \rightarrow [0, 1, 2, 3]$

"hell"

default values & Rules

ending idx of string

start idx of string

$[start: end: step]$

opt opt opt ← optional

+1

when step +ve $\rightarrow (start < stop) \quad (start \neq step)$

←

when step -ve $\rightarrow (start > stop) \quad (start \neq step)$

Syntaxes:

s = "hello"

1) s[: :] → "hello"

2) s[: 3 :] → "hel"

3) s[2 : :] → "llo"

4) s[: : -1] → [-1 : -5 : -1] ⇒ "olleh"

ASCII values :

→ Default unq int values of characters

→ total 256 values are there

a - z → 97 - 122

A - Z → 65 - 90

'0' - '9' → 48 - 57

Type Conversion

ASCII $\rightarrow$ char

`chr ( ASCII ) → char`

char $\rightarrow$ ASCII

ord(char) $\rightarrow$ ASCII

String traversal:

1) for i in str:

print(i) ← value traversal

2) for i in range(len(str))

`print( S[i] )` → travel by idx

Membership and Identity operator usage
(in, not in) (is, is not)

$s = \text{"hello"} \quad a = \text{"he"} \quad b = \text{"hi"}$

• If $(a \text{ in } S)$



Compares

⁴hello⁰¹

"he"

```
return true
```

if ($b \in S$)



4 hello 01

"he"

returns False

$a = \text{"hi"}$

$b = \text{"hi"}$

$c = \text{"ho"}$

